



COMILLAS

UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Desarrollo de aplicaciones y servicios

Documentación sprint 3 - NaiveBay

Sergio Jiménez Romero

Pablo Tuñón Laguna

Tareas a realizar

Durante este sprint se ha trabajado en consolidar y evolucionar la arquitectura de la aplicación, enfocándose en el desarrollo de una API propia que gestione el backend de la web de subastas, eliminando la dependencia de servicios externos. Paralelamente se han construido nuevas interfaces en React para facilitar una interacción más rica y directa con la información almacenada en la base de datos *PostgreSQL*. A continuación se detalla de forma más precisa y estructurada las tareas llevadas a cabo:

- Creado, almacenado, edición y muestreo de subastas, categorías y pujas. Esto aporta el core de funcionalidad a la página web.
- Desarrollo de un CRUD de subastas y pujas en React para una cómoda interacción entre el usuario y los datos de la API.
- Creación de un servicio de registro y acceso de usuario, asociando a los usuarios (almacenados y no repetidos) distintos tokens jwt que permiten el acceso a distintas partes de la web así como distintas funcionalidades.
- Permitir al usuario consultar la información asociada con su cuenta en una interfaz agradable. Cabe destacar que tanto esta tarea como la anterior anteriormente funcionaban con la API de backend: <https://das-p2-backend.onrender.com/api/users/login/> y ahora funcionan con la implementada.
- Almacenado y muestreo de las N subastas que puede tener un usuario, las N pujas que puede realizar y el objeto al que pertenece cada subasta. El almacenado se realiza en Django mientras que el muestreo con una aplicación de React.
- Establecimiento de roles en el backend que permiten realizar unas peticiones u otras en función del usuario que trata de hacerlas (identificado por el token JWT del registro o acceso a la página). Estos roles, que no son más que niveles de seguridad facilitan el control del flujo de datos, permiten la monitorización de usuarios e incitan a aquellos que consultan la página a registrarse y proporcionar sus datos para acceder a un mayor número de funcionalidades.

Asignación de tareas

Nuevamente, no se dispone de un sprint Backlog que permita al equipo de Scrum organizar qué tareas puede escoger y cuales no, sin embargo, en este sprint se disponía de 6 guiones que orientaban el desarrollo de backend. Estos guiones se realizaron de forma conjunta ambos miembros del equipo, ya que proporcionan una base para entender el funcionamiento de las categorías, pujas y subastas así como unos principios para la autorización de servicios en función del rol de usuario. Para las otras tareas que debían ser completadas la división fue la siguiente:

- Sergio Jiménez realizó los filtros para poder acceder a subastas en función de distintos valores, ya fuera una cadena en el título o descripción, un precio... Esto permitió una conexión con los filtros ya desarrollados a lo largo del Sprint 1. Además, se encargó de desarrollar el React necesario para tener un CRUD de subastas y para que cada usuario pueda consultar las subastas o pujas que ha colgado o realizado. Cabe destacar que actualizó las conexiones que se establecieron en el Sprint 2 entre la antigua API de backend y la desarrollada en el Sprint actual, uniendo además las nuevas funcionalidades implementadas.
- Pablo Tuñón implementó el backend necesario para un correcto funcionamiento de las pujas, permitiendo su muestreo, edición, almacenamiento, borrado y creación. También estableció relaciones entre usuarios, pujas y subastas desarrollando también las urls que permitirían un muestreo de dichas subastas y pujas asociadas a cada usuario. Pablo también realizó un testeo intensivo de los distintos servicios del Backend. Encontró una pequeña incongruencia que se menciona en el siguiente apartado, sin embargo, por si se desea, se dejan los dos usuarios empleados en el testeo de permisos y funcionalidades:
 - Usuario: *"User2"* Contraseña: *"User123!"*
 - Usuario administrador: *"ADMIN2"* Contraseña: *"p3p1nill*S"*

Dificultades, desviaciones y soluciones

Cabe destacar, que, gracias a los guiones, no han surgido un gran número de dificultades. Únicamente se han afrontado tres problemas:

- Dado que la implementación se ha hecho en GitHub con el objetivo de disponer de un entorno de control de versiones colaborativo y que permita una fácil subida a producción, el equipo introdujo en el repositorio de backend el entorno virtual del servidor de Django. Esto supuso un problema en un inicio al hacer la subida a la nube por la terminación *LF* de los ficheros creados con *WSL* que se actualizaba a *CRLF* al hacer el *push* al origen. Se entiende que en futuras implementaciones no es la decisión más correcta ya que se suben muchos datos (y además ejecutables y binarios) a GitHub. Además, precisamente por el tipo de datos que se subían, corrían el riesgo de corromperse, teniendo que crear el entorno virtual desde cero hasta cinco veces a lo largo del sprint.
- Se ha detectado una cierta inconsistencia, en los guiones proporcionados para entender el funcionamiento de Django, se menciona como sólo los usuarios administradores deberían poder listar los nombres de usuario. Por el contrario, en los requisitos del sprint, se menciona como se debe poder conocer el nombre de los usuarios que realizan pujas o subastas. Dado que se completó el guion primero, se ha optado por sólo permitir a los usuarios administradores listar a los usuarios, pero es sencillo de modificar. La modificación a aplicar en la vista de listado de usuarios sería un permiso *custom* que permita la visualización a todo tipo de usuarios, pero sólo la modificación a los administradores.
- Las autorizaciones y permisos tampoco han sido sencillos de usar ya que han requerido de un extenso testeo y trabajo hasta cumplir con los requisitos del sprint. También ha sido compleja su integración en el frontend aunque finalmente se ha logrado.